

Day 4 Theory: Databases, SQL, Git and GitHub

Workshop: Web Technology, Hands-on Full-Stack Workshop

Audience: BCA Second Year, Mandsaur University

Time for this day: 2 hours in class, plus about 20 minutes of reading before class

How to use this document

Read it once before class. Do not worry if some parts feel new. Write down your questions and bring them to class.

On Day 3 we built a server with Express. Our notes are kept in an array inside the server. When the server stops, the notes are lost. Today we fix this with a **database**. Then we learn **Git**, a tool that saves the history of your code, and **GitHub**, a website where you can keep a copy of your code.

1. Recap of Day 3

- A **server** waits for requests and sends responses.
 - **Express** is a package that makes servers easier to write.
 - Our **API** has three routes: read notes (public), add a note (password needed), delete a note (password needed).
 - Our notes are kept in **memory** (an array in `data.js`). When the server stops, memory is cleared and the notes are gone.
 - The password is kept in the `.env` file, not in the code.
-

2. What is a database?

A **database** is an organised place to store data, so that the data is **kept** after a program stops, and so that a program can **find** data quickly.

Think of a paper register in an office. The register keeps its pages when the office closes at night. A database does the same job for a program.

Memory and disk

	Memory (RAM)	Disk (a file on the hard drive)
What is kept	Only while the program runs	Kept after the program stops
Our notes on Day 3	Here	Not here
Our notes today	Not only here	Here, inside the database file

A database stores its data on the **disk**. That is why the notes survive a restart.

Tables, rows and columns

Most databases store data in **tables**. A table looks like a sheet in a spreadsheet.

- A **table** is a group of data of one kind. Ours is called `notes`.
- A **column** is one kind of information. Our columns are `id`, `title`, `text` and `created_at`.
- A **row** is one item. One row is one note.

id	title	text	created_at
1	Welcome	This note is stored in a SQLite database.	(date and time)
2	Study plan	Learn HTML on Monday...	(date and time)
3	Idea	Add your own notes...	(date and time)

The primary key

Every row needs a value that is **different for every row**, so that we can point to exactly one row. This is the **primary key**. In our table it is `id`. When we delete a note, we say "delete the note with id 2".

We let the database choose the id by itself. It counts up for each new note. This is called **AUTOINCREMENT**.

3. SQL: the language of databases

SQL (said "S-Q-L" or "sequel") stands for **Structured Query Language**. It is the language we use to talk to most databases. A **query** is a question or an instruction that we send to the database.

SQL words are usually written in capital letters. This is a habit, not a rule. It makes SQL easy to read.

CREATE TABLE: make a table

```
CREATE TABLE IF NOT EXISTS notes (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  title TEXT NOT NULL,
  text TEXT NOT NULL,
  created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP
)
```

- `IF NOT EXISTS` means: do nothing if the table is already there. This lets us run the same code every time the server starts.
- `INTEGER` is a whole number. `TEXT` is text.
- `NOT NULL` means the column must have a value. It cannot be empty.
- `DEFAULT CURRENT_TIMESTAMP` means: if we do not give a value, the database writes the current date and time.

INSERT: add a row

```
INSERT INTO notes (title, text) VALUES ('Welcome', 'Hello')
```

We name the columns, then give the values in the same order. We do not give `id` or `created_at`. The database fills them in.

SELECT: read rows

```
SELECT id, title, text FROM notes
```

This returns every row, with three columns. `SELECT *` means "all columns".

WHERE: pick only some rows

```
SELECT title FROM notes WHERE id = 2
```

`WHERE` is a **filter**. Only the rows that match are used.

ORDER BY: put rows in order

```
SELECT id, title, text FROM notes ORDER BY id DESC
```

`DESC` means **descending**: the biggest `id` first. Because new notes get bigger ids, this puts the **newest note first**. `ASC` means ascending (smallest first).

DELETE: remove rows

```
DELETE FROM notes WHERE id = 2
```

Be careful. If you forget the `WHERE` part, `DELETE FROM notes` removes **every** row.

Summary

What we want	SQL
Make a table	<code>CREATE TABLE</code>
Add a row	<code>INSERT INTO</code>
Read rows	<code>SELECT</code>
Pick some rows	<code>WHERE</code>
Sort rows	<code>ORDER BY</code>
Remove rows	<code>DELETE FROM</code>

4. Never glue user text into SQL: the `?` placeholder

Suppose a user types a note title, and we build the SQL by joining text:

```
// DANGEROUS. Do not write code like this.
const sql = "INSERT INTO notes (title, text) VALUES ('" + title + "', '" + text + "');"

```

If the user types a title that contains a quote mark and SQL words, the database may read the **user's text as part of the SQL instruction**. The user can then change what the instruction does, for example delete data. This attack is called **SQL injection**. It is one of the most common and most harmful web security problems.

It is the same idea as XSS on Day 2: **text from a user was treated as code**.

The fix: placeholders

We write the SQL with a `?` where a value will go. We give the real values **separately**.

```
db.prepare("INSERT INTO notes (title, text) VALUES (?, ?)").run(title, text);
```

The database receives the instruction and the values as **two separate things**. It never reads the values as SQL. Whatever the user types is only data.

Rule: In our code, every value that came from a user goes in through a `?`. We never join user text into a SQL string.

(We also use `?` for the id in `DELETE FROM notes WHERE id = ?`. The id also comes from the browser, so it is user input too.)

5. SQLite: a database in one file

There are many database programs. Some run as a **separate server program** (for example PostgreSQL, which is Track B). **SQLite** is different. It is a small database that lives **inside your own program**. All the data is kept in **one file**.

	SQLite	A separate database server (for example PostgreSQL)
Where it runs	Inside our Node.js program	As its own program on a computer
Data	One file, for us <code>notes.sqlite</code>	Managed by the database server
What to install	Nothing for us	A program to install and start
Good for	Learning, small apps, one server	Bigger apps and many servers

For this workshop, SQLite is the best choice because there is nothing to install.

Built into Node.js

Node.js 24 has SQLite **built in**. We load it like this:

```
const { DatabaseSync } = require("node:sqlite");
```

The name `node:sqlite` means "a module that comes with Node.js". We do not run `npm install` for it.

Important: We checked this on Node.js 24.21.0 on Linux: it worked without any flag and printed no warning. It is **not yet tested on Windows**. If you see a warning that says `ExperimentalWarning`, a warning is not an error. Tell your instructor and see the Stuck Sheet.

The database file is not code

The file `notes.sqlite` holds **data**, not code. We do not upload it to GitHub (see section 8). It is created automatically the first time the server runs.

6. Our new data.js

On Day 3 we promised: "the three function names stay the same". Today we keep that promise. The server calls the same functions as before. Only the **inside** changes.

Function	What it does	SQL inside
<code>init()</code>	New today. Makes the table, and adds 3 starter notes if the table is empty	<code>CREATE TABLE IF NOT EXISTS ,</code> <code>SELECT COUNT(*) , INSERT</code>
<code>listNotes()</code>	Gives back all notes, newest first	<code>SELECT ... ORDER BY id DESC</code>
<code>addNote(title, text)</code>	Saves a note and gives it back with its new id	<code>INSERT ... VALUES (?, ?)</code>
<code>deleteNote(id)</code>	Removes a note. Gives back <code>true</code> or <code>false</code>	<code>DELETE ... WHERE id = ?</code>

Three small ideas to remember:

- `db.prepare(sql)` gets a SQL instruction ready. Then we run it with `.run(...)` (for INSERT and DELETE), `.all()` (for many rows) or `.get()` (for one row).
- `result.changes` tells us how many rows a DELETE removed. If it is more than 0, the note existed.
- `async` and `await` are still used, so that `server.js` works in the same way if we change to a different database later (Track B does this).

Why `init()` and `await`?

The server must **prepare the database before it starts listening**. In `server.js` we write:

```
init().then(function () {
  app.listen(PORT, function () { ... });
});
```

`init()` takes some time and gives back a **promise**: "I will finish later." `.then(...)` means "when it has finished, run this function". So the server starts listening only after the table is ready.

When do the 3 starter notes appear?

`init()` adds the starter notes only if the table has **no rows**. On the very first run, the table is empty, so you get 3 notes. After that, your own notes stay. If you delete **all** the notes and restart the server, the table is empty again, so the 3 starter notes come back. This is how our code is written. It is not a mistake.

Relative file name

We open the database with the name `notes.sqlite`, with no folder. The file is made in the **current folder of the terminal**, the folder where you ran `node server.js`. Always start the server from inside the `notes-board` folder. If you start it from another folder, you get a new empty database there.

7. Version control and Git

The problem

Have you ever saved files like `project-final.zip`, `project-final2.zip` and `project-final-REAL.zip`? This is hard to manage. You do not know what changed, and you cannot easily go back.

What is version control?

Version control is a system that records the **history** of your files. You can see what changed, when, and why. You can go back to an older version. Many people can work on the same project.

Git is the most used version control tool. It runs on your own computer.

Key words

Word	Meaning
Repository (repo)	A project folder that Git is tracking, with its whole history
Commit	One saved snapshot of your files, with a short message that says what you did
Staging	Choosing which changes will go into the next commit
Branch	A line of work. Our main line is called <code>main</code>
Remote	A copy of the repository on another computer, for example on GitHub
Push	Send your commits to the remote

The commands we use

Command	What it does
<code>git init</code>	Starts tracking the current folder. It makes a hidden <code>.git</code> folder
<code>git config user.name "Your Name"</code>	Tells Git your name, for this project
<code>git config user.email "you@example.com"</code>	Tells Git your email, for this project
<code>git status</code>	Shows what changed and what Git is not tracking
<code>git add .</code>	Stages all changes (the dot means "everything here")
<code>git commit -m "message"</code>	Saves the staged changes as a commit
<code>git log --oneline</code>	Shows the list of commits
<code>git branch -M main</code>	Names the current branch <code>main</code>

The flow is always the same: **change files, then `git add`, then `git commit`.**

Git is not GitHub

- **Git** is a tool on your computer.
- **GitHub** is a website that stores a copy of a Git repository. It also makes it easy to share code and to connect to hosting services.

You can use Git without GitHub. Tomorrow we need GitHub, because the hosting service will read our code from there.

8. What must NOT go into Git

Some files must **never** be committed.

File or folder	Why not
<code>.env</code>	It holds secrets (our password). Anyone who can see the repository could read it
<code>node_modules/</code>	It holds downloaded packages. They are large, and anyone can get them again with <code>npm install</code>
<code>*.sqlite</code> (our database file)	It is data, not code. It would put private data online and it changes all the time

We list these names in a file called `.gitignore`. Git does not track a file that matches a line in `.gitignore`.

```
node_modules/
.env
*.sqlite
*.sqlite3
*.db
```

The `*` means "anything". So `*.sqlite` means any file that ends in `.sqlite`.

How do we prove it works? We run `git status`. A file that is ignored does not appear in the list. After we push, we also look at the repository page on GitHub to make sure `.env` is not there. This is the **most important check** of the day.

`.env.example` is different

`.env.example` shows **which settings are needed**, with a fake value. It is safe to commit. A person who gets your code copies it to `.env` and writes the real value.

What if I commit a secret by mistake?

Git keeps the **history**. If you commit `.env` and push it, the secret is in the history, even if you delete the file in the next commit. **Do not count on hiding it again.** What to do:

1. Tell your instructor.
2. **Change the password** (the secret). The old one must be treated as public.

This is why we check `git status` before every first commit.

9. GitHub

GitHub is a website for storing Git repositories online. You need a free account. Creating the account is part of your homework.

Steps (we do them in the lab)

1. On github.com, click **New** to create a **repository**. Give it a name. Make it **empty**: do not add a README or other files.
2. On your computer, connect your project to it with a **remote** called `origin` :

```
git remote add origin https://github.com/YOUR-USERNAME/notes-board.git
```

3. Send your commits:

```
git push -u origin main
```

`-u` remembers the link, so later you can type only `git push` .

4. Refresh the repository page on GitHub. Your files are there.

Signing in

GitHub needs to know who you are when you push. Git for Windows may open a **browser window** and ask you to sign in to GitHub. **This has not been tested on the lab computers**. If sign in fails, your instructor will help. See the Stuck Sheet.

10. Optional Track B: PostgreSQL

This part is for students who finish early. **It is optional. Day 5 works without it.**

PostgreSQL is a database that runs as a **separate server program**. Our Node.js program connects to it over a **connection string** (an address with a user name and a password).

	SQLite (Track A)	PostgreSQL (Track B)
Install	Nothing	A separate program
Our code	<code>node:sqlite</code>	The <code>pg</code> package (<code>npm install pg</code>)
Placeholder	<code>?</code>	<code>\$1</code> , <code>\$2</code>
Where the address is kept	A file name	<code>DATABASE_URL</code> in <code>.env</code>

The idea is the same: tables, rows, columns and SQL. Only the connection and a few words change. The file `optional-postgres/data.js` has the same four function names, so `server.js` does not change.

We tested this version of `data.js` on PostgreSQL 16 on Linux only. **It is not tested on Windows.**

11. Quick revision

1. A **database** keeps data after the program stops.
2. A **table** has **columns** and **rows**. A **primary key** (`id`) is different for every row.
3. **SQL**: `CREATE TABLE` , `INSERT` , `SELECT` , `WHERE` , `ORDER BY` , `DELETE` .
4. Use `?` placeholders. **Never join user text into SQL**. That is how SQL injection happens.
5. **SQLite** is one file, and Node.js 24 has it built in (`node:sqlite`).
6. `data.js` keeps the same function names, and adds `init()` . `server.js` waits for `init()` before it listens.
7. **Git** records the history of your project: `init` , `add` , `commit` , `log` .
8. Never commit `.env` , `node_modules` or `*.sqlite` . Use `.gitignore` , and check with `git status` .
9. **GitHub** stores a copy online. Connect with `git remote add origin` , send with `git push -u origin main` , and check that `.env` is **not** there.
10. PostgreSQL (Track B) is a separate server program. It is optional.

12. Homework for Day 5

1. Make sure your **GitHub account works**. Sign in at github.com and write down your **username**. **You need your GitHub login on Day 5.**
2. Read about **hosting** (see the reading list below).
3. Run your Notes Board and check that your notes stay after you restart the server.
4. Write down at least two questions for Day 5.

Reading list: hosting

Tomorrow we put the Notes Board on the internet. Before class, find answers to these questions in your own words. A web search is fine.

1. What is **web hosting**? What is the difference between your computer (`localhost`) and a hosted server?
 2. What is a **public URL**?
 3. Why does a hosting service need your code? How can it get it from GitHub?
 4. Some hosting services do not keep files that a program writes while it runs. If this were true, what would happen to a SQLite file? (We will check this on Day 5.)
-

Glossary

Word	Meaning
AUTOINCREMENT	The database counts up and gives each new row the next id
Branch	A line of work in Git. Ours is <code>main</code>
Column	One kind of information in a table
Commit	One saved snapshot of the project in Git
Database	An organised place to store data so that it is kept
<code>.gitignore</code>	A file that lists what Git must not track
Git	A tool that records the history of your files
GitHub	A website that stores Git repositories online
Placeholder (?)	A mark in SQL where a value goes. The value is sent separately
PostgreSQL	A database that runs as a separate server program
Primary key	A column that is different for every row, such as <code>id</code>
Promise	A result that will be ready later
Push	Send commits to a remote
Query	An instruction or a question sent to a database
Remote	A copy of the repository on another computer
Repository	A project folder that Git tracks, with its history
Row	One item in a table. For us, one note
Secret	Private data such as a password. It must not be in Git
SQL	Structured Query Language, the language of databases
SQL injection	An attack where user text is read as SQL
SQLite	A small database that keeps all its data in one file
Staging	Choosing which changes go into the next commit
Table	A group of rows of one kind